

Publier et vendre des extensions Chrome sans appel API depuis l'extension

Procédure complète pour NoXoZ.be - Chrome Web Store, paiement externe, licence locale

ChatGPT - synthèse technique

28 juin 2026

Table of Contents

Decision rapide

Tu veux publier plusieurs extensions orientées ChatGPT sur le Chrome Web Store. Tu veux les vendre autour de 3 à 5 euros. Tu ne veux pas que l'extension fasse un appel API vers Gumroad, Lemon Squeezy, Dodo Payments, Stripe, Paddle, ExtensionPay ou ton propre serveur.

La bonne architecture est donc celle-ci.

L'extension reste gratuite sur le Chrome Web Store. Le paiement se fait en dehors du Chrome Web Store. L'utilisateur paie sur une page externe. Il reçoit une clé de licence. Il colle cette clé dans l'extension. L'extension valide la clé localement. Aucun appel réseau n'est fait pour valider la licence.

Point important. Le Chrome Web Store ne gère plus les paiements natifs pour les extensions. Chrome Web Store Payments a été arrêté pour les nouveaux achats. Donc il faut passer par un paiement externe.

Recommandation nette

Pour ton cas, je recommande ceci.

Phase 1. Publie les extensions gratuitement sur le Chrome Web Store. Ajoute un écran de licence local. Vends les licences avec une page externe simple. Génère les clés toi-même avec un script local. Envoie la clé au client par email. Ça permet de démarrer sans backend.

Phase 2. Quand il y a assez de ventes, automatise la génération de clés avec un petit système externe. Ce système peut être déclenché par webhook après paiement. Mais l'extension, elle, ne contacte toujours rien. Elle valide seulement la signature locale.

Phase 3. Si le volume devient serieux, tu peux ajouter un portail client, une gestion de refunds et une revocation. Mais ce n'est pas necessaire pour vendre une extension a 3, 4 ou 5 euros.

Les quatre extensions concernees

D'apres le contexte fourni, les extensions visees sont les suivantes.

1. SOLOStarter. Elle ouvre un nouveau chat ChatGPT et injecte un fichier de regles.
2. ChatGPT Export. Elle exporte le chat courant en Markdown local.
3. NoXoZ_AI_VoiceControl. Elle pilote le flux voix vers texte, auto-send, lecture a haute voix et relance.
4. AutoSend ou VoiceFlow selon le repo exact. Elle automatise l'envoi apres transcription et peut enchaîner avec la lecture vocale.

Le meme modele de licence peut etre utilise pour les quatre. Chaque extension doit avoir son propre identifiant produit dans la licence. Exemple.

- noxoz-solostarter
- noxoz-chatgpt-export
- noxoz-ai-voicecontrol
- noxoz-autosend

Budget reel

Chrome Web Store

Google demande un frais unique de developpeur Chrome Web Store. Le montant public historiquement documente est 5 dollars US. C'est un paiement unique pour ouvrir le compte developpeur. Google indique aussi une limite standard de 20 extensions publiees par compte avant demande d'augmentation.

Paielement externe

Le cout principal n'est pas Google. Le cout principal est la plateforme de paiement.

Pour des prix bas comme 3 euros, le forfait fixe de 40 ou 50 centimes fait mal. A 3 euros, 50 centimes represente deja 16,7 pourcent du prix avant meme le pourcentage.

Conclusion commerciale. Ne vends pas a 2 euros. Evite 3 euros si tu peux. Prix conseille. 4,99 euros. Ou 5,99 euros si l'extension fait gagner du temps tous les jours.

Comparatif simple des plateformes

Gumroad

Avantages. Tres rapide a lancer. Bon pour vendre un petit produit numerique. Page produit simple. Gestion fiscale annoncee comme Merchant of Record depuis 2025. Generation de cles possible.

Inconvenients. Frais eleves pour petits montants. La validation officielle des cles Gumroad passe normalement par leur API. Si tu refuses tout appel API depuis l'extension, tu ne dois pas utiliser leur validation API dans l'extension.

Verdict. Bon pour demarrer vite. Moins bon si tu veux optimiser les frais.

Lemon Squeezy

Avantages. Merchant of Record. Frais de base plus bas que Gumroad. Gestion des licences et cles logiciel disponible. Bon choix pour vendre du logiciel numerique.

Inconvenients. La validation standard des licences Lemon Squeezy utilise leur License API. Donc, pour ton exigence stricte, il ne faut pas faire appeler cette API par l'extension. Il faut utiliser ta propre cle signee localement, ou automatiser seulement cote serveur apres paiement.

Verdict. Bon equilibre pour vendre a l'international. Bon candidat apres Gumroad si tu veux moins de frais.

Dodo Payments

Avantages. Merchant of Record. Prix affiche de base plus bas. Cles de licence integrees. Livraison automatique. API et webhooks.

Inconvenients. La validation robuste reste normalement basee sur leur systeme de licence. Si tu veux zero appel API depuis l'extension, tu dois utiliser Dodo uniquement pour le paiement et la livraison, ou pour declencher un generateur externe.

Verdict. Interessant si tu veux un systeme plus oriente SaaS/licensing. A tester sur un produit pilote avant de migrer les quatre extensions.

Paddle

Avantages. Merchant of Record solide. Tres bon pour SaaS, abonnements, TVA internationale, factures, support fiscal.

Inconvenients. Trop lourd pour des extensions a 3 ou 5 euros au lancement. Setup plus oriente business SaaS que petit produit simple.

Verdict. Pas le premier choix pour ton cas au depart.

Stripe direct

Avantages. Controle total. Frais de paiement souvent bas. Tres flexible.

Inconvenients. Stripe n'est pas automatiquement un Merchant of Record classique pour ton business. Tu dois gérer davantage la TVA, les factures, les remboursements, la conformité et la logique de licence. Il faut plus de développement.

Verdict. Pas idéal si tu veux démarrer vite et vendre à petit prix sans infrastructure.

ExtensionPay

Avantages. Fait pour les extensions Chrome. Intègre paiement et contrôle d'accès.

Inconvenients. Le modèle standard implique du code d'intégration dans l'extension et une vérification externe. C'est exactement ce que tu veux éviter.

Verdict. Pas adapté à ta contrainte zéro appel API.

Choix final pour toi

Le plus simple. Gumroad ou Lemon Squeezy pour encaisser. Ton propre script local pour générer les clés. Validation locale par signature cryptographique dans l'extension.

Le plus propre techniquement. Lemon Squeezy ou Dodo Payments pour encaisser. Webhook vers un petit générateur de licence. Email automatique de la clé. Validation locale dans l'extension avec clé publique.

Le meilleur compromis maintenant. Commence manuel. Ne construis pas de backend tout de suite. Vends la première version. Si les ventes existent, automatise.

Pourquoi une clé locale simple ne suffit pas

Une clé de type checksum est facile à faire. Exemple. On prend un email, un produit et une date. On calcule un hash. On fabrique une clé. L'extension refait le calcul. Si le hash correspond, elle accepte.

Problème. Pour recalculer un hash secret, l'extension doit contenir le secret. Or l'utilisateur peut inspecter le code d'une extension. Donc le secret finit par être extractible.

La meilleure solution locale est une signature asymétrique.

Tu signes la licence avec une clé privée. La clé privée reste chez toi. L'extension contient seulement la clé publique. L'extension peut vérifier la signature. Elle ne peut pas générer de nouvelles licences.

C'est exactement ce qu'il faut pour ton cas.

Architecture de licence recommandee

Principe

Une licence est un texte compose de deux parties.

Premiere partie. Un payload JSON encode en base64url.

Deuxieme partie. Une signature cryptographique du payload.

Format conseille.

NOXOZ1.<payload_base64url>.<signature_base64url>

Le payload contient par exemple.

```
{  
  "product": "noxoz-ai-voicecontrol",  
  "email_sha256": "hash_email_client",  
  "order": "reference_paiement",  
  "iat": "2026-06-28",  
  "exp": null,  
  "tier": "standard",  
  "license_id": "NVC-2026-000001"  
}
```

L'extension verifie seulement ceci.

1. Le format commence par NOXOZ1.
2. Le payload est lisible.
3. Le champ product correspond a l'extension.
4. La signature est valide avec la cle publique integree dans l'extension.
5. La date d'expiration est absente ou future.
6. Le tier permet les fonctions demandees.

Aucun fetch. Aucun appel API. Aucun serveur contacte par l'extension.

Limites de securite a accepter

Il faut etre lucide. Une extension Chrome est du code cote client. Un utilisateur motive peut toujours modifier le code localement. Il peut retirer le test de licence. Il peut patcher une version pirate.

Mais pour une extension a 5 euros, ce n'est pas ton vrai probleme. Ton vrai probleme est de rendre l'achat simple, propre et credible.

Le systeme par signature locale est suffisant pour bloquer les usages non techniques et les clés inventées. Il ne bloque pas un pirate motivé. Aucun système purement local ne bloque totalement un pirate motivé.

Procédure complète Chrome Web Store

Étape 1. Créer le compte développeur

Va dans le Chrome Web Store Developer Dashboard. Connecte-toi avec le compte Google que tu veux utiliser pour publier. Idéalement, utilise une adresse dédiée à NoXoZ.be. Accepte le Developer Agreement. Paye le frais unique d'inscription. Complète le profil développeur.

À préparer.

- Nom public du développeur.
- Email support.
- Site web ou page support.
- Politique de confidentialité.
- Moyen de paiement pour le frais développeur.

Conseil. N'utilise pas une adresse Gmail personnelle bordelique. Utilise une adresse dédiée au projet. Exemple. support@noxoz.be ou une adresse Google dédiée à la publication.

Étape 2. Préparer le dossier extension

Le ZIP doit contenir le fichier manifest.json à la racine. Pas dans un sous-dossier.

Correct.

```
mon-extension.zip
manifest.json
background.js
content.js
popup.html
popup.js
styles.css
icons/icon16.png
icons/icon48.png
icons/icon128.png
```

Incorrect.

```
mon-extension.zip
mon-extension/
  manifest.json
  background.js
```

Le deuxième cas peut provoquer une erreur du type manifest introuvable.

Etape 3. Nettoyer le ZIP

Le ZIP ne doit pas contenir les fichiers de dev inutiles.

A exclure.

- .git/
- .github/
- .old/
- .tmp/
- .private/
- .docs/
- node_modules/ si non nécessaire
- README brouillon
- CHANGELOG privé
- captures privées
- archives zip anciennes
- fichiers de test
- logs
- secrets
- certificats
- .env
- *.pem
- *.key

A inclure.

- manifest.json
- scripts runtime nécessaires
- HTML/CSS nécessaires
- icônes
- assets UI nécessaires
- _locales/ si internationalisation
- LICENSE si tu veux
- README public si utile, mais pas obligatoire dans l'extension

Etape 4. Tester localement

Dans Chrome ou Brave.

Ouvre.

chrome://extensions

Active le mode developpeur. Clique sur Charger l'extension non empaquee. Selectionne le dossier de l'extension. Teste tout. Regarde les erreurs dans Inspect views, service worker et console de la page cible.

Checklist de test.

- L'extension se charge sans erreur.
- Le popup s'ouvre.
- Les boutons fonctionnent.
- Les permissions demandees sont justifiees.
- Aucune erreur rouge dans la console.
- Aucun appel externe non voulu.
- Le mode licence accepte une vraie cle.
- Le mode licence refuse une fausse cle.
- Le mode gratuit reste utilisable si prevu.

Etape 5. Versionner correctement

Chaque upload doit avoir une version plus haute que la precedente dans manifest.json.

Exemple.

"version": "1.0.0"

Puis.

"version": "1.0.1"

Ne reutilise pas une version deja publiee.

Exemple de manifest MV3

Exemple generique pour une extension ChatGPT. A adapter a chaque extension.

```
{
  "manifest_version": 3,
  "name": "NoXoZ AI VoiceControl",
  "version": "1.0.0",
  "description": "Voice control workflow for ChatGPT with local settings and optional license unlock.",
  "icons": {
    "16": "icons/icon16.png",
    "48": "icons/icon48.png",
    "128": "icons/icon128.png"
  },
  "action": {
```

```

    "default_title": "NoXoZ AI VoiceControl",
    "default_popup": "popup.html"
  },
  "background": {
    "service_worker": "background.js"
  },
  "permissions": [
    "storage",
    "activeTab",
    "scripting"
  ],
  "host_permissions": [
    "https://chatgpt.com/*",
    "https://chat.openai.com/*"
  ],
  "content_scripts": [
    {
      "matches": [
        "https://chatgpt.com/*",
        "https://chat.openai.com/*"
      ],
      "js": ["content.js"],
      "css": ["styles.css"],
      "run_at": "document_idle"
    }
  ]
}

```

Remarque. Ne demande pas all_urls si chatgpt.com suffit. Moins tu demandes de permissions, moins tu augmentes le risque de rejet ou de review longue.

Politique reseau zero API

Dans ton code, evite tout appel reseau externe.

A chercher avant publication.

```

fetch(
XMLHttpRequest
axios
$.ajax
WebSocket
EventSource
chrome.runtime.sendMessage

```

importScripts avec URL externe
<script src="https://...">

Si tu trouves un appel externe non indispensable, retire-le.

Dans Manifest V3, il faut aussi éviter le code distant. Le code exécute doit être dans le package de l'extension. Pas chargé depuis un CDN.

Ecran licence dans l'extension

Interface minimale.

Licence NoXoZ

Collez votre clé de licence.

[champ texte]

[Valider localement]

Etat : version gratuite

ou

Etat : licence valide pour NoXoZ AI VoiceControl

Stockage.

Utilise chrome.storage.local. Ne stocke pas de secret privé. Stocke seulement la clé utilisateur et l'état local calculé.

Exemple.

```
await chrome.storage.local.set({
  licenseKey: key,
  licenseStatus: "valid",
  licenseProduct: payload.product,
  licenseTier: payload.tier
});
```

Génération locale de licence

Méthode conseillée

Utilise une signature ECDSA P-256 ou Ed25519. Pour compatibilité WebCrypto dans Chrome, ECDSA P-256 est un choix pratique.

La clé privée sert à signer. Elle reste hors extension. La clé publique sert à vérifier. Elle est intégrée dans l'extension.

Process manuel au debut

Quand tu recois une vente.

1. Tu prends l'email client.
2. Tu prends le produit achete.
3. Tu prends l'ID de commande.
4. Tu lances ton generateur local.
5. Tu copies la cle genere.
6. Tu l'envoies au client.

C'est manuel. C'est lent. Mais c'est parfait pour tester le marche sans serveur.

Process automatise plus tard

Quand une vente arrive.

1. Gumroad, Lemon Squeezy ou Dodo envoie un webhook.
2. Ton petit generateur recoit l'evenement.
3. Il signe une licence.
4. Il envoie la cle par email.
5. L'extension valide localement.

Dans ce schema, il y a un serveur externe. Mais ce serveur est uniquement pour vendre et envoyer la cle. L'extension ne l'appelle jamais.

Exemple de commande pour zipper proprement

Depuis le dossier de l'extension.

```
rm -f ../NoXoZ_AI_VoiceControl-v1.0.0-cws.zip
zip -r ../NoXoZ_AI_VoiceControl-v1.0.0-cws.zip . \
-x "*.git*" \
-x "*.zip" \
-x "*.pem" \
-x "*.key" \
-x "*.env" \
-x "node_modules/*" \
-x ".old/*" \
-x ".tmp/*" \
-x ".private/*" \
-x ".docs/*" \
-x "logs/*"
```

Verification.

```
unzip -l ../NoXoZ_AI_VoiceControl-v1.0.0-cws.zip | head -50
```

Tu dois voir manifest.json directement, sans dossier parent.

Exemple de script generateur de licence

Ce bloc est un exemple de principe. Il faut le garder hors extension. Il sert cote createur, pas cote client.

```
#!/usr/bin/env python3
```

```
import base64
```

```
import hashlib
```

```
import json
```

```
from datetime import date
```

```
from cryptography.hazmat.primitives import hashes, serialization
```

```
from cryptography.hazmat.primitives.asymmetric import ec
```

```
from cryptography.hazmat.backends import default_backend
```

```
def b64url(data: bytes) -> str:
```

```
    return base64.urlsafe_b64encode(data).rstrip(b"=").decode("ascii")
```

```
def canonical_json(obj: dict) -> bytes:
```

```
    return json.dumps(obj, sort_keys=True, separators=(",", ":")).encode("utf-8")
```

```
def email_hash(email: str) -> str:
```

```
    return hashlib.sha256(email.strip().lower().encode("utf-8")).hexdigest()
```

```
def load_private_key(path: str):
```

```
    with open(path, "rb") as f:
```

```
        return serialization.load_pem_private_key(f.read(), password=None,  
backend=default_backend())
```

```
def make_license(private_key_path: str, product: str, email: str, order: str, license_id: str):
```

```
    private_key = load_private_key(private_key_path)
```

```
    payload = {
```

```
        "product": product,
```

```
        "email_sha256": email_hash(email),
```

```
        "order": order,
```

```
        "iat": str(date.today()),
```

```

    "exp": None,
    "tier": "standard",
    "license_id": license_id,
  }
  payload_bytes = canonical_json(payload)
  signature = private_key.sign(payload_bytes, ec.ECDSA(hashes.SHA256()))
  return "NOXOZ1." + b64url(payload_bytes) + "." + b64url(signature)

if __name__ == "__main__":
  key = make_license(
    private_key_path="noxoz_private_key.pem",
    product="noxoz-ai-voicecontrol",
    email="client@example.com",
    order="ORDER-12345",
    license_id="NVC-2026-000001",
  )
  print(key)

```

Creation d'une cle privée et publique.

```

openssl ecparam -name prime256v1 -genkey -noout -out noxoz_private_key.pem
openssl ec -in noxoz_private_key.pem -pubout -out noxoz_public_key.pem
chmod 600 noxoz_private_key.pem

```

La cle privée ne va jamais dans GitHub. La cle privée ne va jamais dans le ZIP Chrome Web Store. La cle privée ne va jamais dans l'extension.

Validation locale cote extension

Principe JavaScript.

```

function b64urlToBytes(input) {
  const pad = "=".repeat(((4 - input.length % 4) % 4));
  const base64 = input.replace(/-/g, "+").replace(/_/g, "/") + pad;
  const raw = atob(base64);
  return Uint8Array.from([...raw].map(ch => ch.charCodeAt(0)));
}

async function importPublicKey(pem) {
  const clean = pem
    .replace("-----BEGIN PUBLIC KEY-----", "")
    .replace("-----END PUBLIC KEY-----", "")
    .replace(/\s+/g, "");
}

```

```

const der = Uint8Array.from(atob(clean), c => c.charCodeAt(0));

return crypto.subtle.importKey(
  "spki",
  der.buffer,
  { name: "ECDSA", namedCurve: "P-256" },
  false,
  ["verify"]
);
}

async function verifyLicense(licenseKey, expectedProduct, publicKeyPem) {
  const parts = licenseKey.trim().split(".");
  if (parts.length !== 3 || parts[0] !== "NOXOZ1") {
    return { valid: false, reason: "format_invalid" };
  }

  const payloadBytes = b64urlToBytes(parts[1]);
  const signatureBytes = b64urlToBytes(parts[2]);
  const payloadText = new TextDecoder().decode(payloadBytes);
  const payload = JSON.parse(payloadText);

  if (payload.product !== expectedProduct) {
    return { valid: false, reason: "wrong_product" };
  }

  if (payload.exp && new Date(payload.exp) < new Date()) {
    return { valid: false, reason: "expired" };
  }

  const publicKey = await importPublicKey(publicKeyPem);
  const ok = await crypto.subtle.verify(
    { name: "ECDSA", hash: "SHA-256" },
    publicKey,
    signatureBytes,
    payloadBytes
  );

  if (!ok) {
    return { valid: false, reason: "bad_signature" };
  }
}

```

```
return { valid: true, payload };  
}
```

Attention technique. Les signatures ECDSA produites par OpenSSL/Python sont souvent en DER. WebCrypto attend parfois un format brut r+s selon les API. Il faut tester le format exact. Si besoin, il faut convertir la signature DER en 64 octets r+s avant stockage. Alternative plus simple. Utiliser une petite librairie de verification ECDSA incluse localement dans l'extension, sans CDN.

Publication dans le Developer Dashboard

Upload

Dans le dashboard. Clique sur Add new item. Choisis le ZIP. Upload. Si le manifest et le ZIP sont valides, Google cree l'item.

Store Listing

A remplir.

- Nom de l'extension.
- Summary courte.
- Description complete.
- Categorie.
- Langue.
- Icone.
- Screenshots.
- Eventuels visuels promotionnels.
- URL support.
- URL privacy policy.

Pour les screenshots, Google recommande au moins une capture, et idealement jusqu'a cinq. Les tailles recommandees sont 1280 x 800 ou 640 x 400.

Privacy

A remplir proprement.

- Single purpose.
- Justification de chaque permission.
- Declaration des donnees collectees.
- Declaration remote code.
- URL privacy policy.

Pour ton modele zero API, tu dois pouvoir declarer que l'extension n'execute pas de code distant. Il faut que ce soit vrai dans le code.

Distribution

Choisis la visibilite.

Public. L'extension est visible dans le Store.

Unlisted. Seulement accessible avec le lien. Pratique pour tester proprement avant diffusion large.

Private. Surtout pour organisation ou domaine. Pas ton cas principal.

Pour le lancement, je recommande Unlisted d'abord. Tu testes avec quelques personnes. Ensuite tu passes Public.

Test instructions

Si l'extension a besoin d'un compte, d'un site precis ou d'un mode de test, explique-le clairement.

Exemple.

Open <https://chatgpt.com/>.

Click the extension icon.

Use the free mode without license.

To test license unlock, use this test key: XXXXX.

No external license validation call is performed by the extension.

The license is verified locally with a public key embedded in the extension package.

Politique de confidentialite minimale

Il te faut une page privacy policy. Tu peux l'heberger sur noxoz.be ou GitHub Pages.

Modele de contenu.

Privacy Policy - NoXoZ AI VoiceControl

This extension runs locally in the user's browser.

It is designed for ChatGPT pages.

It stores user settings locally using chrome.storage.local.

It does not sell user data.

It does not send license keys to an external license server.

It does not execute remote code.

If voice or transcript features are used, the extension interacts with the current ChatGPT web page selected by the user.

NoXoZ.be does not receive the user's prompts, transcripts, conversations, or license checks from the extension.

Support contact: support@noxoz.be

Adapte ce texte pour chaque extension. Ne mens pas. Si une extension collecte ou envoie quelque chose, il faut le déclarer.

Description Store conseillée

SOLOStarter

Summary.

Create a new ChatGPT chat and inject your own rules file quickly.

Description courte.

SOLOStarter helps advanced ChatGPT users start a clean new chat with predefined rules or workflow instructions. It is built for users who repeatedly work with structured prompts, local rules and technical workflows.

ChatGPT Export

Summary.

Export the current ChatGPT conversation locally as Markdown.

Description courte.

ChatGPT Export lets you export the current conversation to a local Markdown file. It is designed for users who need clean local archives, technical notes, logs or documentation from ChatGPT sessions.

NoXoZ AI VoiceControl

Summary.

Control ChatGPT voice workflows with transcription, auto-send and read aloud actions.

Description courte.

NoXoZ AI VoiceControl adds workflow controls around voice transcription, auto-send and read aloud actions in ChatGPT. It is designed for power users who want a faster hands-free interaction loop.

AutoSend ou VoiceFlow

Summary.

Automate ChatGPT send and read-aloud flow after voice transcription.

Description courte.

This extension automates selected ChatGPT voice workflow actions after transcription. It is built for users who want a smoother voice-to-text-to-response loop in the browser.

Modele gratuit plus licence

Je recommande de ne pas bloquer tout le produit. Une extension totalement bloquee donne moins confiance.

Modele conseille.

Version gratuite.

- Fonctions de base.
- Test manuel.
- Limite raisonnable.
- Message clair pour debloquer.

Version licence.

- Auto flow complet.
- Presets avances.
- Export complet.
- Boucle continue.
- Options avancees.

Exemple.

Free mode: manual controls.

Licensed mode: full automation and advanced presets.

Prix conseille

Pour lancement.

- 4,99 euros paiement unique par extension.
- 12,99 euros pack trois extensions.
- 14,99 euros pack quatre extensions.

Evite 2 euros. Ce prix donne une impression de gadget et les frais fixes mangent trop la marge.

Prix psychologique correct. 4,99 euros. C'est assez bas pour un achat impulsif. C'est assez haut pour absorber les frais.

Checklist avant publication

Technique

- manifest_version vaut 3.
- manifest.json est a la racine du ZIP.
- version augmentee.
- icones presentes.
- pas de fichier secret.
- pas de .git.
- pas de cle privee.
- pas de CDN.
- pas de remote code.
- permissions minimales.
- host_permissions limitees a ChatGPT si possible.
- test local OK.
- licence locale testee avec vraie cle.
- fausse cle refusee.
- extension utilisable en free mode.

Store

- Nom clair.
- Summary sous 132 caracteres.
- Description propre.
- Screenshots propres.
- Icone 128.
- Support URL.
- Privacy policy URL.
- Justification permissions.
- Remote code declare correctement.
- Data usage coherent.
- Test instructions pour reviewer.

Commercial

- Page produit Gumroad, Lemon Squeezy ou Dodo.
- Prix choisi.
- Email support.
- Procedure d'envoi de cle.
- Modele de mail client.
- Politique de remboursement simple.
- Suivi ventes dans un fichier local.

Mail client pour envoyer la cle

Bonjour,

Merci pour votre achat.

Voici votre cle de licence NoXoZ.

Produit: NoXoZ AI VoiceControl

Cle: NOXOZ1.xxxxxx.yyyyyy

Pour l'activer:

1. Ouvrez ChatGPT dans Chrome ou Brave.
2. Cliquez sur l'icone de l'extension.
3. Ouvrez l'onglet Licence.
4. Collez la cle.
5. Cliquez sur Valider localement.

La validation est faite localement dans votre navigateur.
L'extension ne contacte pas de serveur de licence externe.

Support: support@noxoz.be

NoXoZ.be

Risques de rejet Chrome Web Store

Permissions trop larges

Probleme. Tu demandes all_urls alors que tu n'as besoin que de ChatGPT.

Correction. Utilise seulement.

```
"host_permissions": [  
  "https://chatgpt.com/*",  
  "https://chat.openai.com/*"  
]
```

Description trop vague

Probleme. La description dit seulement "best AI extension".

Correction. Explique la fonction exacte.

Remote code

Probleme. Un script est charge depuis un CDN.

Correction. Inclure la librairie localement dans le ZIP.

Donnees utilisateur mal declarees

Probleme. L'extension lit du contenu de page, stocke des options ou manipule du texte, mais la privacy policy ne dit rien.

Correction. Declare precisement ce qui est lu, stocke, envoye ou non envoye.

Fonction payante mal expliquee

Probleme. L'utilisateur installe l'extension et ne comprend pas ce qui est gratuit ou payant.

Correction. Afficher clairement Free mode et Licensed mode.

Plan en une heure

Minute 0 a 10

Choisir l'adresse developpeur. Ouvrir le Chrome Developer Dashboard. Payer les frais developpeur. Completer le profil.

Minute 10 a 20

Choisir une extension pilote. Je recommande ChatGPT Export ou SOLOStarter. Elles sont plus simples a expliquer et moins sensibles que VoiceControl.

Minute 20 a 35

Nettoyer le repo. Creer le ZIP. Tester l'extension en local. Verifier manifest.json.

Minute 35 a 45

Creer les visuels minimum. Icone 128. Une a trois captures d'ecran. Page privacy policy simple.

Minute 45 a 55

Uploader dans Developer Dashboard. Completer Store Listing. Completer Privacy. Choisir Unlisted. Ajouter test instructions.

Minute 55 a 60

Relire. Soumettre pour review. Noter l'ID de l'extension. Preparer la page de paiement externe.

Ordre de lancement conseille

1. ChatGPT Export. Simple, utile, faible risque.
2. SOLOStarter. Simple a expliquer aux power users.
3. AutoSend. Plus technique, mais utile.
4. NoXoZ AI VoiceControl. Plus complexe, plus sensible, plus de cas de test.

Ce qu'il ne faut pas faire

Ne mets pas la cle privée dans l'extension. Ne mets pas un secret HMAC dans l'extension si tu veux une vraie generation impossible cote client. Ne fais pas de fetch vers Gumroad ou Lemon Squeezy si ton engagement public est zero API. Ne charge pas de code depuis un CDN. Ne demande pas all_urls sans raison. Ne publie pas avec des screenshots flous. Ne vends pas a 2 euros. Ne promets pas une securite anti-piratage totale.

Conclusion operationnelle

Tu peux publier rapidement. Le compte Chrome Web Store est simple. Le ZIP doit etre propre avec manifest.json a la racine. Le paiement doit etre externe. La licence doit etre locale si tu refuses les appels API.

Le schema technique le plus propre est une licence signee. Cle privée chez toi. Cle publique dans l'extension. Validation locale. Aucun appel reseau.

Pour commencer, fais manuel. Une vente arrive. Tu generes une cle. Tu l'envoies. Tu observes. Si ca vend, tu automatises seulement le back-office. L'extension reste offline pour la licence.

Sources consultees

Chrome for Developers - Register your developer account. Chrome for Developers - Prepare your extension. Chrome for Developers - Publish in the Chrome Web Store. Chrome for Developers - Manifest file format. Chrome for Developers - Privacy fields. Chrome Web Store payments deprecation. Gumroad pricing and license key documentation. Lemon Squeezy pricing and License API documentation. Dodo Payments pricing and License Keys documentation. Paddle Merchant of Record documentation. Stripe pricing documentation.

Sources consultees le 28 juin 2026.